Play-Type Markov Chain v2

One-pager · Smart Stadium VIP · October 14, 2026

The v2 Markov chain predicts the type of the upcoming play (run, pass, field goal, punt or other) from its pre-snap situation, using no video. Its probabilities are multiplied with the video classifier's to give the final prediction (paper Sections 4.2–4.3).

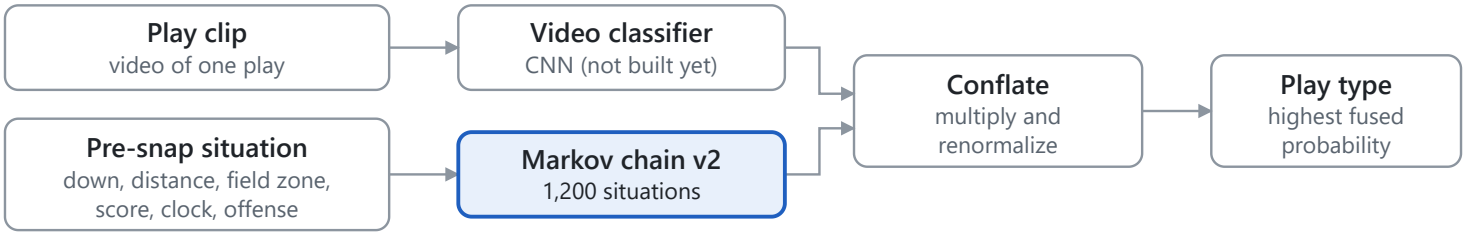
64.8% v1: 54.3%
Markov chain alone, all plays

80.3% v1: 67.3%
Markov chain alone, Georgia Tech offense

87.3% v1: 84.8%
Full model, simulated 71% classifier

2017 FBS season (869 games), 5-fold cross-validation by game; full-model figure: one 80/20 split, simulated classifier. Data: cfbfastR play-by-play.

How it fits into the full model



Both models output probabilities over the five play types; `conflate_probabilities` multiplies them. Highlighted: this package.

How to use it

Unzip `markov_chain_v2.zip` (or use the repo) and run `pip install -r requirements.txt`. `download_cfb_pbp` fetches a season of play-by-play once (about 72 MB) into `data/cfb_pbp/`. Fit on past plays, then ask about any pre-snap situation:

```
from playByPlay import download_cfb_pbp, load_cfb_pbp
from markovChain import PlayMarkovChain
plays = load_cfb_pbp(download_cfb_pbp(2017))
mc = PlayMarkovChain().fit(plays)
situation = dict(offense='Georgia Tech', down=3, distance=8,
    yards_to_goal=65, score_diff=0, half=1, period=2,
    seconds_left_half=600)
mc.next_play_type_proba(situation)
# -> [0.27, 0.70, 0.00, 0.00, 0.03] for R, P, F, U, O
mc.describe(situation)  # bucket for each indicator
```

`notebooks/conflatedPlayPrediction.ipynb` runs the whole workflow in about 3 minutes; `python -m pytest tests -q` runs the 25 tests.

Plugging it in

1. **Classifier:** run the play-type CNN on the clip (not in the repo yet; `preSnapDetection.ipynb` already finds each clip's pre-snap frame). Output order: R, P, F, U, O.
2. **Situation:** offense, down, distance, yards to goal, score difference, half, quarter, seconds left in the half: from the play-by-play for recorded games, the scoreboard for live ones.
3. **Markov chain:** call `next_play_type_proba` on that situation.
4. **Conflate:** multiply the two vectors; the highest value is the predicted play type.

```
from markovChain import PlayMarkovChain, team_run_rates
from conflation import conflate_probabilities
CLASSES = ('R', 'P', 'F', 'U', 'O')  # classifier's order
# train on earlier games; team tendency from last season
mc = PlayMarkovChain().fit(
    past_plays, run_rates=team_run_rates(last_season_plays))
def predict_play(clip_probs, situation):
    mc_probs = mc.next_play_type_proba(
        situation, play_types=CLASSES)
    fused = conflate_probabilities([clip_probs, mc_probs])
    return CLASSES[fused.argmax()], fused
```

Prediction indicators: before vs now

v1 predicted from the play that just happened; v2 predicts from the situation the offense faces before the snap, and adds three indicators.

Indicator	Before (v1)	Now (v2)
Down	Previous play's down	Upcoming play's down, 1st–4th
Distance to go	Not used	1, 2–3, 4–6, 7–10, 11+ yards
Field position	Previous play's, 10-yard buckets	Upcoming play's: red zone, opp 21–35, opp 36–49, own side, own deep
Previous play type	Core of the state	Off by default (optional)
Team tendency	Not used	Run rate: pass-heavy (<45%), balanced, run-heavy (60%+)
Score and clock	Not used	Normal, trailing late, leading late, end of 1st half
Situations modelled	280 states, 78,400 transitions	1,200 situations
Rare situations	Near-uniform guess	Fall back to a coarser situation
Play types predicted	7, incl. PAT and kickoff	5, matching the classifier
Trained on	Simulated games	869 real FBS games (2017)

- **Down matters most.** Dropping it raises log loss from 0.723 to 0.969; field zone (0.769) and distance (0.756) come next.
- **Previous play type is off:** 7× the situations for a 0.001 log-loss gain.
- **Team tendency** barely moves overall accuracy (64.2% to 64.8%) but lifts Georgia Tech from 71.8% to 80.3%.

Open items before integration

- [ ] **Classifier output:** produce real CNN probabilities in R, P, F, U, O order. The 87.3% above uses a simulated classifier.
- [ ] **Clip-to-play matching:** clip names carry the year, game code and quarter (e.g. `2017_G6016_Q4_012`). A table mapping game codes to play-by-play games is needed to look up each clip's situation.
- [ ] **"Other" class:** in the play-by-play data it means pre-snap fouls; confirm the video labels mean the same, not ads or replays.
- [ ] **More training data:** add the 2014–2016 seasons (`load_cfb_pbp` takes a list), and use the previous season's run rates for any game being predicted.